

More Information

Design reasoning behind parts of the Equipment Cleaning Log that a reader may reasonably question

Field	Detail
Prepared by	Aravind R · aravind.r194@gmail.com
Prepared for	Leucine — TNN Technologies Private Limited, Bangalore
Companion to	Design & Requirements Specification, README.md, NOTES.md
Date	09 September 2026

This document covers areas where the implementation makes a choice that is not self-evident from reading the code, and states the reasoning plainly — including where a decision was deliberately left incomplete. It is written to be read alongside the specification rather than instead of it.

1. Roles and the Seeded Accounts

1.1 Why roles exist at all

This comes from GMP practice, and it is the entire reason the users table carries a role column: the person who does the work should not be the person who attests that it was done.

An operator cleans a piece of equipment and logs it. A supervisor independently confirms it. In a regulated plant that separation is the point of a second signature — without it the record is one person vouching for themselves, which is not evidence.

The term for this is segregation of duties. It is a domain requirement rather than a user-interface preference, which is why it is enforced in the service layer and not merely reflected in the UI.

1.2 What each role can do

Role	Can do	Cannot do
operator	Sign in; create, read and edit equipment and cleaning records	Move a cleaning record to verified
supervisor	Everything an operator can, plus verify	—
admin	Everything a supervisor can	Nothing — identical to supervisor today (see 1.4)

That is the complete picture. Everything other than verification requires only an authenticated session; no role gates creating equipment, retiring it, or editing cleaning records.

1.3 What is enforced, and where

There is exactly one role check in the codebase. It is defined once:

```
// api/src/domain/types.ts
export const VERIFIER_ROLES: readonly UserRole[] = ['supervisor', 'admin'];
```

and consumed in exactly one place:

```
// api/src/services/cleaningRecords.ts – resolveStatusTransition()
if (!VERIFIER_ROLES.includes(actor.role)) {
  throw new ForbiddenError('Only a supervisor or admin can verify a cleaning record.');
```

WHY THE SERVICE LAYER, NOT ROUTE MIDDLEWARE

Verification is not a separate endpoint. It is PATCH /api/cleaning-records/:id with { "status": "verified" } — the same endpoint that changes the method or the notes.

Middleware runs before the request body is interpreted, so it cannot distinguish a verification from an ordinary edit without parsing the body itself. Placing the check where the status transition is decided means every caller of that function is covered, including any route added later.

The front-end also hides the Verify action from operators, but that is a convenience rather than a control: it avoids offering an action that would always fail. The server refuses regardless of what the interface shows, and an integration test signs in as an operator, attempts a verification, and asserts the 403.

1.4 A deliberate gap: admin does nothing different

STATED PLAINLY

The admin role behaves exactly like supervisor. It exists in the enum so that a permission matrix has somewhere to land later; it earns nothing today.

This is a scope decision, not an oversight. A full role-based permission matrix is listed under exclusions in NOTES.md, and inventing a distinction such as "admin can delete users" would have been building a feature the brief did not ask for. The one role rule the domain genuinely needs is who may verify, and that is enforced and tested.

1.5 The seeded accounts

All four share the password password123, printed in the README so that a reviewer can sign in within a minute of starting the application. They are created by `api/scripts/seed.ts` and hashed with Node's built-in `crypt`.

Email	Name	Role	Why this account exists in the seed
priya@leucine.test	Priya Nair	operator	Author of most seeded cleaning records. Use this account to see the 403 on an attempted verification.
arun@leucine.test	Arun Menon	operator	A second operator, so records are not all authored by one person and "cleaned by" is visibly a real reference rather than a constant.
ravi@leucine.test	Ravi Kumar	supervisor	The verifier on every seeded verification. This is what makes segregation of duties visible in the audit trail itself: the verify event is by a different person than the create event.
divya@leucine.test	Divya Rao	admin	Present only to exercise the third role. Owns no seeded data.

Signing in as Ravi gives the fullest picture: he is the only seeded account that can verify and also has history attributed to him.

1.6 Where role appears across the stack

Layer	File	What it does with role
Database	<code>migrations/002_enums.sql</code>	Declares <code>user_role</code> as a native enum, so an invalid role cannot be stored at all.
Domain	<code>domain/types.ts</code>	Declares the role union and <code>VERIFIER_ROLES</code> — the single definition of who may verify.
Session	<code>auth/session.ts</code>	Places the role into the signed session claims, so it returns on every request without a database lookup.
Service	<code>services/cleaningRecords.ts</code>	The only enforcement point. Refuses a verification attempted by a non-verifier.
Front-end	<code>web/src/App.tsx</code>	Decides whether the Verify control renders. Presentation only.

A trade-off worth naming: the role travels inside the session token rather than being read per request. That saves a query on every call, at the cost of staleness — a role change does not take effect until the session expires, which is eight hours here. If roles changed frequently the right answer would be a shorter token lifetime, or reading the role per request and accepting the query.

1.7 Questions this design invites, and the answers

Question	Answer
Why does admin exist if it does nothing different?	It is an extension point, and today it is identical to supervisor. A full permission matrix was out of scope and is listed under exclusions. The one role rule the domain needs — who may verify — is enforced and tested. One real check is worth more than three roles with invented differences.
Why is the check in the service rather than middleware?	Verification is not its own endpoint; it is a PATCH with status: verified, the same endpoint that edits the method or notes. Middleware runs before the body is interpreted and cannot tell them apart. Enforcing at the transition means no future route can bypass it.
The interface hides the button — is that not enough?	No. That is convenience, not a control: the API can be called directly. The server returns 403 regardless, and a test asserts it.
Where does the audit actor come from? Could a client forge it?	From the signed session token, never from the request body. A test sends a body naming a different actor and asserts the recorded actor is still the session user. Request schemas are also strict, so an actor field in a body is rejected as an unknown key rather than silently ignored.
What happens if a role changes mid-session?	Nothing until the session expires, because the role is carried in the token. See the trade-off noted in 1.6.
Why can a verified record not be un-verified?	It is a one-way transition; an attempt returns 422. In a regulated workflow a mistaken verification is corrected by recording a new cleaning, not by rewriting the previous attestation. This is a documented assumption rather than a requirement of the brief.

Further topics will be added to this document as they arise — the audit diff and canonical value rendering, keyset pagination and the cursor, the transaction and row locking, the error envelope, and the choice of raw SQL over an ORM are all recorded in NOTES.md in the meantime.